ADVANCE SELENIUM

Contents

- TestNG
- Framework
- Maven
- Extent Reports
- Coding Standards
- GitHub
- Jenkins

TestNG

- TestNG stands for “Test Next Generation”.
- It is a Unit testing framework tool basically used by developers to perform unit testing
- We Automation Engineers use TestNG for
    1. Batch execution
    2. Parallel execution
    3. Group execution
    4. Report generation

Installation

1. In eclipse go to Help and select Eclipse Marketplace.
2. Type testNG in search textbox and click on the search button.
3. Click on the install button present under testNG for eclipse.
4. Click on confirm.
5. Click on radio button “I agree to license agreement”.
6. Click on next and click on install anyway on the popup.
7. Click on finish and restart now.

Note :- After the installation, add TestNG library to the java project as well.

- Right click on java project , go to Build path and select Add Libraries.
- Select testNG and click on Next and Finish.
- It will display the TestNG folder in the java project.

Rules to follow while using TestNG

1. Use **@Test** instead of main method.
2. Use **Reporter.log()** instead of System.out.println().

@Test

- Annotation is Java template which is used to give information to compiler and run timeenvironment.
- Annotations can be used on top of variable, method or a class
- @Test is a core annotation of TestNG without which we cannot run any TestNG programs
- Any class which contains at least one @Test annotated method is called **TestNG class**.
- Any method which is annotated with @Test annotation is considered as **Test Case** and code written within this method is considered as **Test Steps**.

Note :- TestNG runs based on Alphabetical order.

Annotations Attributes

1. priority

- It determines the sequence of execution of the test cases.
- The lowest priority test case will run first, and the highest priority test case will be executed last.
- If priority is not specified, then the default priority will be 0.

  Ex : @Test(priority = -1)

Note :-

- we go for priority to execute the test method in required order.
- If priority is duplicated, then it will be executed in Alphabetical order.

2. invocationCount

It is used when we want to run same test case multiple times.
By default, value of invocationCount is 1.
Ex :- If we want to run single @Test for 10 times

@Test(invocationCount = 10)

3. threadPoolSize

- It is used for parallel execution
- When we want to execute same test case multiple times in parallel we go for threadPoolSize.
- It works only when invocationCount attribute is used.
- By default threadPoolSize is 0.

Ex :- @Test (invocationCount = 10 , threadPoolSize = 5)

4. enabled

- It contains the boolean value
- By default, It's value is "true". If you want to skip some test cases, then you need to explicitly specify "false" value.

Ex :- @Test(enabled = false)

5. dependsOnMethods

- It will wait till the dependent method is executed successfully and then only executes that particular method.
- The scope of dependsOnMethods is within particular testNG class.

EX:

```
@Test(dependsOnMethods = "createAccount")
public void deleteAccount() {
    Reporter.log("Account deleted succesfully");
}
@Test()
public void createAccount() {
    Reporter.log("Account created succesfully");
}
```

6. dependsOnGroups

- It is used to perform group execution at project level.
- First we have to group the test cases by using an attribute called "groups".
- This feature help in executing set of groups to be executed before a test method.
- Scope is throughout the project.

Batch Execution

Executing multiple test classes or test scripts in one shot is called batch execution.

(or)

Executing all the classes together via testng.xml file is called batch execution.

Note :- xml file is also called as suite file

To create suite file :-

1. Select all TestNG classes.
2. Right click, go to testNG and select convert to TestNG.
3. Click on finish. It will create a suite file called testng.xml

Contents in testng.xml file (suite file)

```
<suite name="Suite">
    <test thread-count="5" name="Test">
        <classes>
            <class name="bikes.ToLaunchBajaj" />
            <class name="bikes.ToLaunchTvs" />
            <class name="bikes.ToLauchKTM" />
            <class name="bikes.ToLaunchYamaha" />
        </classes>
    </test>
</suite>
```

To Execute it :-

1. Open the suite file and click on run button

Parallel execution

- Executing multiple test cases simultaneously or parallelly is called parallel execution.
- In xml file at test level provide parallel="classes".

Ex:

```
<suite name="Suite">
    <test thread-count="5" name="Test" parallel="classes">
        <classes>
            <class name="bikes.ToLaunchBajaj" />
            <class name="bikes.ToLaunchTvs" />
            <class name="bikes.ToLauchKTM" />
            <class name="bikes.ToLaunchYamaha" />
        </classes>
    </test>
</suite>
```

Group Execution

- The name of the group is to be mentioned.
- We can group multiple methods under a test suite using xml file.

- The groups should be done inside the test tag before classes tag.
- This helps us to either include or exclude a group.

Ex:

```
<suite name="Suite">
    <test thread-count="5" name="Test" parallel="classes">
        <groups>
            <run>
                <include name="smoke"></include>
            </run>
        </groups>
        <classes>
            <class name="bikes.ToLaunchBajaj" />
            <class name="bikes.ToLaunchTvs" />
            <class name="bikes.ToLauchKTM" />
            <class name="bikes.ToLaunchYamaha" />
        </classes>
    </test>
</suite>
```

Report generation

- Once after the execution, a report will be generated automatically.
- We can able to view the report only when we refresh the project.
- It will be present in test-output folder(emailable-report.html and index.html)

Note :- Every time we run script old execution report will be replaced with new execution report.

Configuration annotations

@BeforeSuite
@AfterSuite
@BeforeTest
@AfterTest
@BeforeClass
@AfterClass
@BeforeMethod
@AfterMethod

@BeforeSuite

- It will run only once for a suite file.
- Here we will write code for creating connection to database and configuring reports.

@AfterSuite

- We will close the connection of database and flush all the reports.

@BeforeTest

- It will be executed based on the number of test runners in a suite file.
- Pre conditions which is common for a particular test runner will be written here.

@AfterTest

- Post conditions which is common for that particular test runners will be written here.

@BeforeClass

- It will execute each class tag in a test runner.

- Here we will write code to launch a browser.

@AfterClass

- Here we will write code for closing the browser and the session.

@BeforeMethod

- It will execute for each test case i.e, for each @Test method.
- Here we will write code for doing login action.

@After Method

- Here we will write code for doing logout.

<u>Chronological order of configuration annotations</u>

```
@BeforeSuite
        @BeforeTest
                @BeforeClass
                        @BeforeMethod
                                @Test( test case TC_001)
                        @AfterMethod
                @AfterClass
        @AfterTest
@AfterSuite
```

Data Provider

- To Execute the same test case multiple times with different set of test data we make use of @DataProvider annotation.
- Sending data from data provider to test case is called dependency injection.

Ex:

```
@DataProvider(name = "credential")
public String[][] testData() {
    String[][] data = { { "admin", "manager" }, { "trainee", "trainee" },};
    return data;
}

@Test(dataProvider = "credential")
public void getData(String[] cred) {
    System.out.println(cred[0]);
    System.out.println(cred[1]);
}
```

Output

```
admin
manager
trainee
trainee
```

Assertion

- Assertions are basically the libraries or functions which are used to verify certain critical checkpoints of test cases.

- Asserts in selenium validate the automated test cases that help testers understand if tests have passed or failed.
- We go for Assertion to reduce lines of code.

Types of Assert

1. Hard Assert
2. Soft Assert

1. Hard Assert

- If there is any failure it will terminate the execution.
- Use assertEquals() present in Assert class.

Syntax :- Assert. assertEquals(expected value , actual value , message);

2. Soft Assert

- If there is any failure it will not terminate the execution.
- Use assetEquals() present in SoftAssert class.

Syntax :- SoftAssert softassert = new SoftAssert();
softassert.assertEquals(expected value , actual value , message);

Note :- At the end of the script call assertAll() to show assertion failure.

Assert (Hard Assert)	Soft Assert
In case of failure, test case will be terminated abruptly	In case of failure, test case will not terminate abruptly
No need to call any method to show assertion error	We need to call assertAll() to show assertion failure
All the methods are static methods	All the methods are non static methods

Cross browser testing/Cross browser parallel testing

- Executing the test cases or test scripts in different browsers is called as cross browser testing.
- It is also known as compatibility testing.
- In TestNG class use @Parameters annotation over the @Test method
- In suite file use <parameter> tag to pass data to TestNG class

```java
public class CrossBrowser {
    WebDriver driver=null;
    @Parameters("Browser")
    @Test
    Run | Debug
    public void browserLaunch(@Optional("chrome") String browser) {
        if(browser.equals("chrome")) {
            driver=new ChromeDriver();
        }else if(browser.equals("firefox")) {
            driver=new FirefoxDriver();
        }else if(browser.equals("edge")) {
            driver=new EdgeDriver();
        }
        driver.get("https://demo.actitime.com/login.do");
    }
}
```

```xml
<suite name="Suite" parallel="tests">
  <test thread-count="5" name="ChromeTest">
      <parameter name="Browser" value="chrome"></parameter>
    <classes>
      <class name="crossbrowser.CrossBrowser"/>
    </classes>
  </test> <!-- Test -->
  <test thread-count="5" name="FirefoxTest">
      <parameter name="Browser" value="firefox"></parameter>
    <classes>
      <class name="crossbrowser.CrossBrowser"/>
    </classes>
  </test>
</suite> <!-- Suite -->
```

Listeners

- It is a feature available in TestNG, which is used to capture run time events during execution and perform appropriate action based on event type.
- @Listeners is TestNG annotation used to monitor the test execution in runtime and generates run time event.

ITestListener

- It is an interface in TestNG used to capture test run time event.
- We should write listener implementation class to use @Listeners annotation.
- Listener implementation class should implements ITestListener and override required abstract method(i.e. onTestFailure, onTestSuccess, onTestStart…)

<u>How to use Listeners?</u>

- Declare @Listeners in every test scripts above the class block and provide listener implementation class.

```java
@Listeners(ListenerImplementation.class)
Run All
public class TestScript {
    @Test
    Run | Debug
    public void login() {
        Reporter.log("Open browser", true);
        Reporter.log("Navigate to url", true);
        Reporter.log("Enter valid credentials", true);
        Reporter.log("Click on login button", true);
        Assert.fail();
    }
}
```

```java
public class ListenerImplementation implements ITestListener,ISuiteListener{

    @Override
    public void onTestStart(ITestResult result) {
        Reporter.log("Test case started", true);
    }

    @Override
    public void onTestSuccess(ITestResult result) {
        Reporter.log("Test case success", true);
    }

    @Override
    public void onTestFailure(ITestResult result) {
        Reporter.log("Test case fail", true);
        // write code to capture screenshot
    }
}
```

- <u>To enable Listener in suite level</u>

1. ListenerImplementation class should implements ISuiteListener.
2. In suite file use <listener> inside <listeners> tag before <test> tag
3. Class-name should be packagename.classname

```xml
<suite name="Suite">
    <listeners>
        <listener class-name="listener.ListenerImplementation"></listener>
    </listeners>
    <test thread-count="5" name="Test">
        <classes>
            <class name="listener.TestScript" />
        </classes>
    </test> <!--
    Test -->
</suite> <!--
```

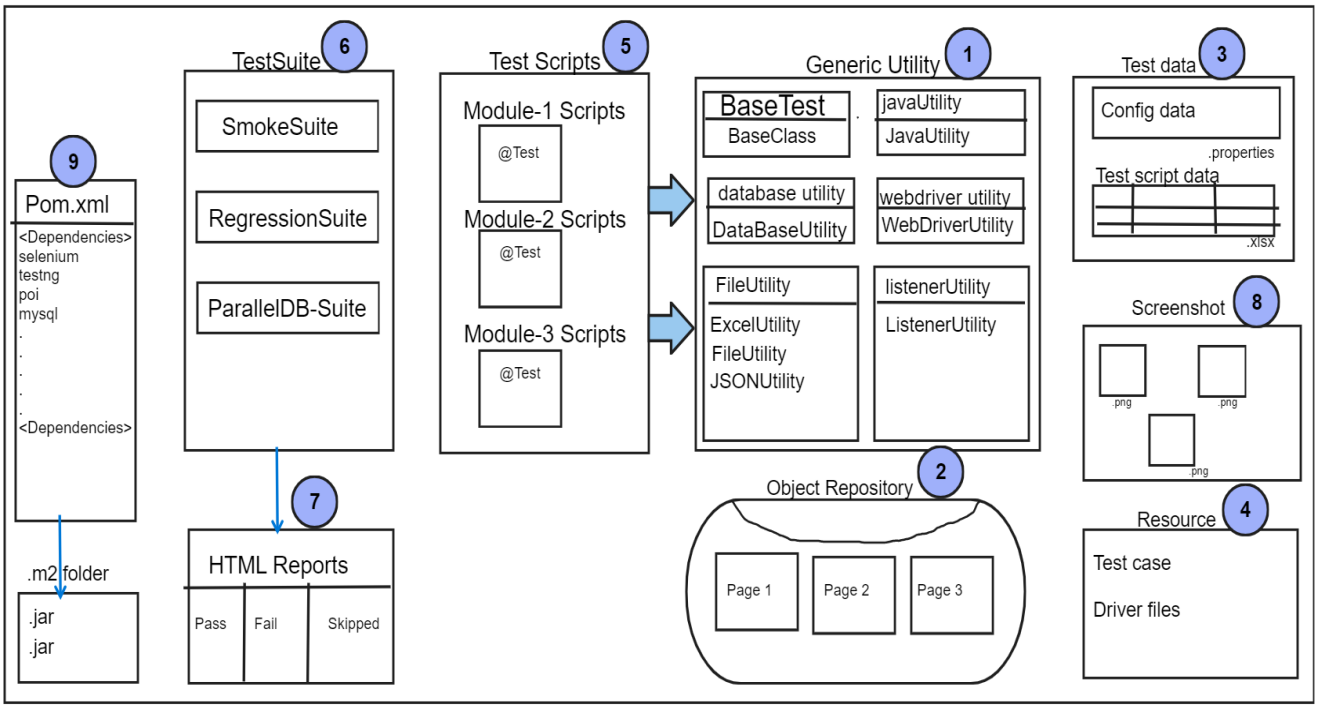
Framework

- Framework is a collection of reusable components that makes automation development , execution and modification easier and faster.
- Framework is a set of rules and guidelines or a best practice to be followed while automating the web application.

Framework Components

1. Generic utility
2. POM/Object repository
3. Test data
4. Resource
5. Test scripts
6. HTML report
7. Screenshots
8. Suite file
9. Pom.xml

Framework Architecture



Framework Explanation in interview

1. Definition
2. Framework Architecture

My Framework components are Generic utility, Object repository, test data , resources ,test scripts, test suite files, HTML reports, Screenshots, pom.xml(Maven build file).

Generic Utility: It is a main component which contains generic classes, generic means which is common for all the project.
JavaUtility: It contains all the java logic methods like generate random number.
DataBaseUtility: To collect data from database we have created method inside this package.
WebDriverUtility: It contains methods like mouse actions, switching to window etc.
FileUtility: It is used to fetch data from external files like property file,excel file,JSON file.
ListenerUtility: It is used to monitor the test run time event and to capture Screenshot for the failed test cases.
Obejct Repository: It is used to store web elements related to project in individual page classes.
Test data: It contains two folders

Configuration data : Used to store all the common data which are useful throughout the project.

Test script data: Used to store the data which are particular to test cases.

Resource: It contains the test cases related to project and the driver files if required.
Test scripts: It contains all the scripts to be executed.
Test suite : suite files are created to perform batch execution , group execution and parallel execution
HTML report: Once after the execution reports will be generated and in that we will get to know how many scripts got pass , fail and skipped.
Screenshot: In this folder we will store all the “failed” script screenshots.
pom.xml : It is used to handle all the dependency jars.

- It will download all the required jars from global to local repo and attach it to the project automatically.
- .m2 folder holds all downloaded jars from maven.

Advantages of framework

- Improve test efficiency.
- Maximum test coverage.
- Test script development is faster and easier because of reusability.
- Test script is optimized.
- Framework provides generic reusable utility for all actions like excel utility, base class , database utility.
- Test script can be used for every new build.

Disadvantages of framework

- To use framework AE should be good at programming.
- AE should have knowledge of multiple third party tools like(testNg ,apache)
- AE should be good at debugging.

Phases of framework

1. Framework design phase
2. Framework implementation phase
3. Framework execution phase

Types of framework

i. Data driven framework
ii. Modular driven framework
iii. Keyword driven framework
iv. Hybrid driven framework
v. Behaviour driven development framework (Cucumber)

Data driven framework

In order to execute test script with multiple inputs or test data , we use an excel and property file , so we call our framework a data driven framework.

Modular driven framework

Since we maintain our framework module wise, we also call our framework a modular driven framework.

Keyword/Method driven framework

In order to avoid writing repetitive steps again and again we use lots of reusable methods and keywords so we call our framework a method driven framework.

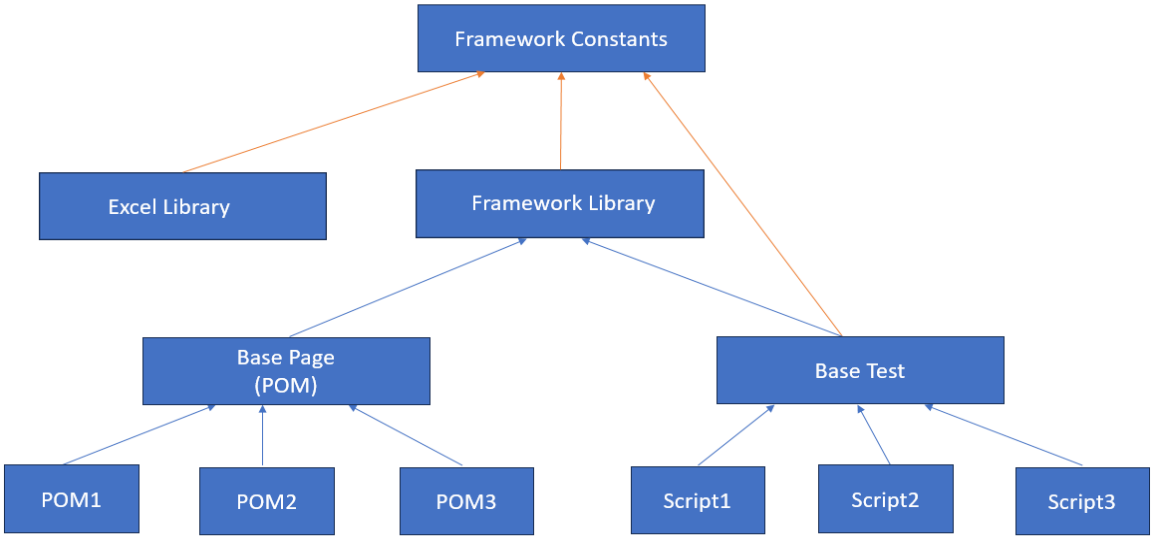
Hybrid driven framework

It is a combination of 2-3 frameworks

Behaviour driven development framework

It is software development approach that allows tester/business analyst to create test cases in simple test language (English).

Hybrid driven framework



Framework constants

- An interface where all framework related constants are declared and initialized.
- E.g :- Path of configuration file , Path of test data file , Path of screenshot directory etc.

Framework Library

- An abstract class where all generic functions are implemented.
- E.g :- Entering value in text field, selecting item from listbox , click element etc.

Excel Library

- A concrete class which contains methods related to reading data from excel.

Base Page

- An abstract class which contains methods/features common to all pages in the application.

Base Test

- An abstract class which contains methods which are reusable by all TestNG classes.
- E.g :- @BeforeSuite , @AfterSuite, @BeforeClass , @AfterClass, @BeforeMethod, @AfterMethod

Maven

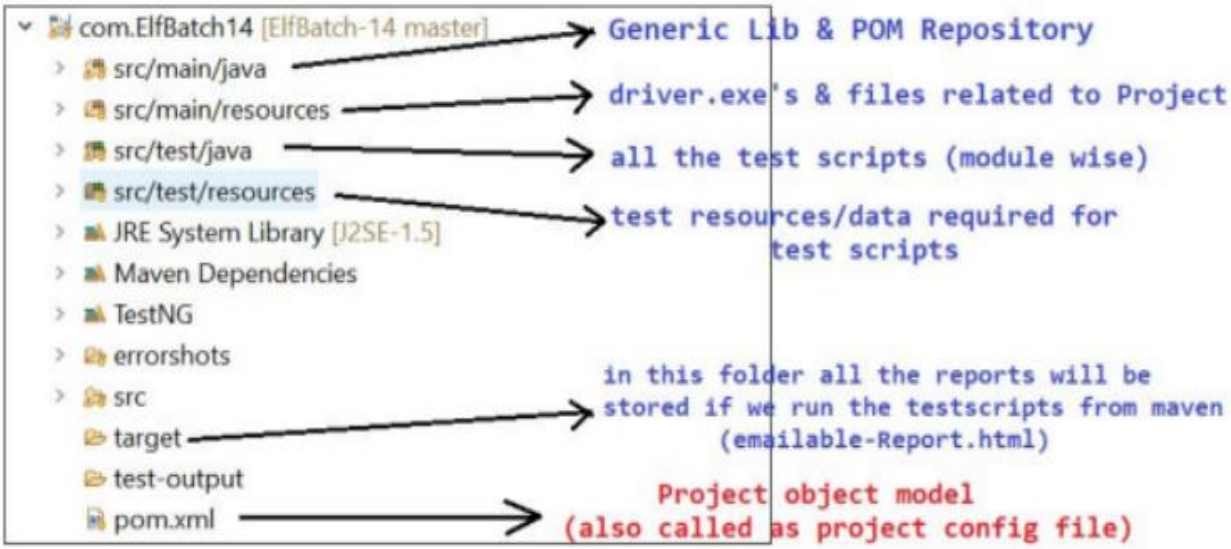
- It is a build testing tool/ build management tool which is used for build creation and build testing and build deployment.
- Developers use this tool for build creation, build installation and build deployment.
- In Automation we use it for build testing and also to handle dependencies.

Build testing :- Maven is used to check the compilation and integration issue between the source code, when multiple developers/automation engineers working on same project and same application.
Build creation :- The process of converting source code format into executable file format is called Build creation.
Build deployment :- Deploy the executable file into the server is called Build deployment process.

Advantages of Maven
- It is used to check the integration and compilation issues between framework components.
- To handle dependency jars.
- To create a quick setup of framework for new engineers.
- To create a framework folder structure.
- We can run the test script in command line without eclipse.
- Maven supports Jenkins.

Maven folder structure
- <u>To create Maven project</u>

Eclipse new Project Maven Project Enter Group ID & Artifact ID Finish



- <u>Installation steps of Maven compiler</u>

```
1. Go to Google and Search for download Maven

2. Click on first link
```

maven.apache.org › download

Maven – Download Apache Maven

4 days ago — Apache **Maven** 3.8.1 is the latest release and recommended version for all users.

The currently selected **download** mirror is https://mirrors.ocf.

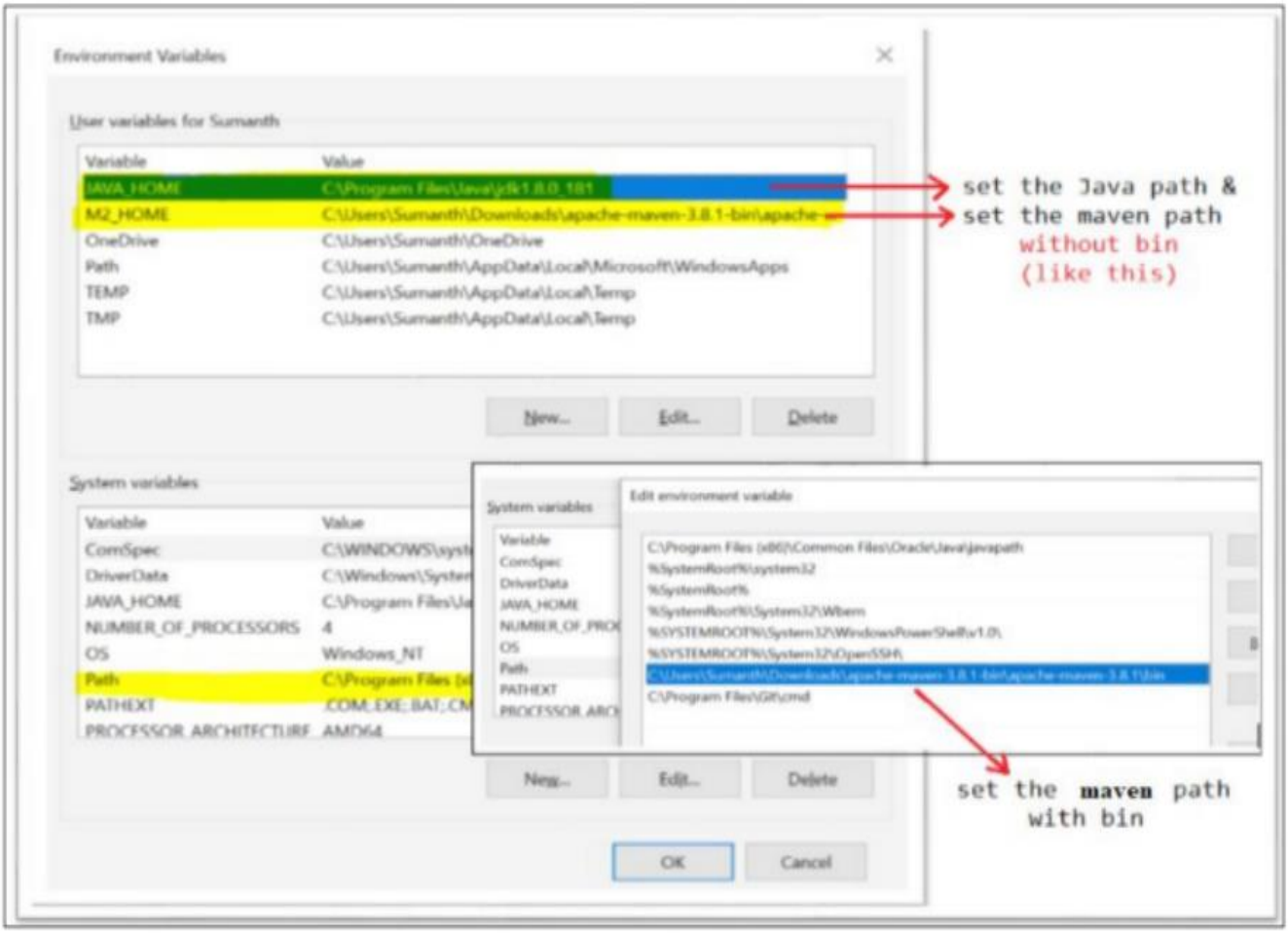
```
3. Click on (Binary zip archive        apache-maven-3.8.1-bin.zip)
```

	Link
Binary tar.gz archive	apache-maven-3.8.1-bin.tar.gz
Binary zip archive	apache-maven-3.8.1-bin.zip

```
4. Go to downloads and extract the Zip file

5. open the file and the bin path location

6. Go to System Environment variable and set the Path
```



Maven dependency

- Dependency is a feature available in maven tool, which is used to download required jar file from global repository(http://mvnrepository.com) to local repository(.m2 folder).
- **Ex:**

```
<dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.11.0</version>
</dependency>
```

How to execute test script from cmd line?

- Project→right click→properties→copy the project location
- Open cmd prompt and change the directory to project directory use "cd" command.
- And start executing maven commands.

Maven Build life cycle/Maven commands

- mvn clean :- Used to clean/delete the reports present in target folder.
- mvn validate :- Used to validate the entire framework.
- mvn compile :- Used to check compilation and integration issue and if any jar is missing it will start downloading the jar automatically.
- mvn test :- Used to execute all the test scripts in src/test/java ,which are suffixed with Test.
- mvn package :- Used to wrap all the source code into .jar or .exe
- mvn install :- Used to install .jar file in different environments/platforms.
- mvn deploy :- Used to deploy the project to production environment.

Maven Surefire Plugin

- It is used to execute testng.xml file via pom.xml.
- In order to achieve this, we need to insert surefire plugin inside pom.xml and mention all the xml files which has to be executed.

Note:- After adding surefire plugin if we run mvn test in cmd line, it will only execute those testing.xml files which is included in pom.xml

Steps to insert surefire plugin inside pom.xml

1. Open browser and search for "maven surefire plugin".
2. Click on first link and go to Examples then click on Using TestNG.
3. Copy the code present in "Using Suite XML Files".
4. Go to pom.xml and use <build> tag then paste inside it.

- In between <suiteXmlFile> tag mention .xml file, which has to be executed.

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.1.2</version>
            <configuration>
                <suiteXmlFiles>
                    <suiteXmlFile>testng.xml</suiteXmlFile>
                </suiteXmlFiles>
            </configuration>
        </plugin>
    </plugins>
</build>
```

Maven Profiling

- Profiling help us to execute specific testng.xml via pom.xml.
- Whenever framework contains multiple suite file or testng.xml file, in order to execute specific suite file via cmd line we go for profiling.

<u>Steps to achieve :-</u>

1. Go to pom.xml and type <profiles> within <project> tag
2. Inside <profiles> tag use <profile> tag
3. Then use <id> and provide unique name to identify.
4. Add surefire plugin.

Syntax :- **mvn test –P id**

Note :- We have to specify which profile we want to run by giving the Id of that profile.

```
<profiles>
    <profile>
        <id>demo</id>
        <build>
            <plugins>
                <plugin>
                    <groupId>org.apache.maven.plugins</groupId>
                    <artifactId>maven-surefire-plugin</artifactId>
                    <version>3.1.2</version>
                    <configuration>
                        <suiteXmlFiles>
                            <suiteXmlFile>testng.xml</suiteXmlFile>
                        </suiteXmlFiles>
                    </configuration>
                </plugin>
            </plugins>
        </build>
    </profile>
</profiles>
```

Maven Parameter

- Maven parameter help us to pass the data from the cmd line without eclipse, So that all the test scripts can be executed based on run time parameter.
- Maven parameter play major role in CI/CD execution.

- Syntax :- mvn test –Dkey=value
- Eg:- mvn test -Durl=https://demo.actitime.com/login.do

How to use parameter in selenium script?
String value= **System.getProperty("Key");**

How to execute only one test using maven cmd?
mvn test –Dtest=test script name

Extent Reports

- ExtentReports is a logger-style reporting library for automated tests.
- Extent reports can be easily integrated with JUnit, NUnit,and TestNG frameworks.
- These reports provide advanced filters and screenshots features that help in analyzing the tests effectively.

Advantages of Extent Reports

- 1. It provides both high level and low-level reports.
- 2. It can be easily integrated with CI/CD pipelines.
- 3. It is an open-source library.
- 4. It provides eye-catching and lucid reports.
- 5. It has good documentation.

Steps to add extent reports dependency in pom.xml

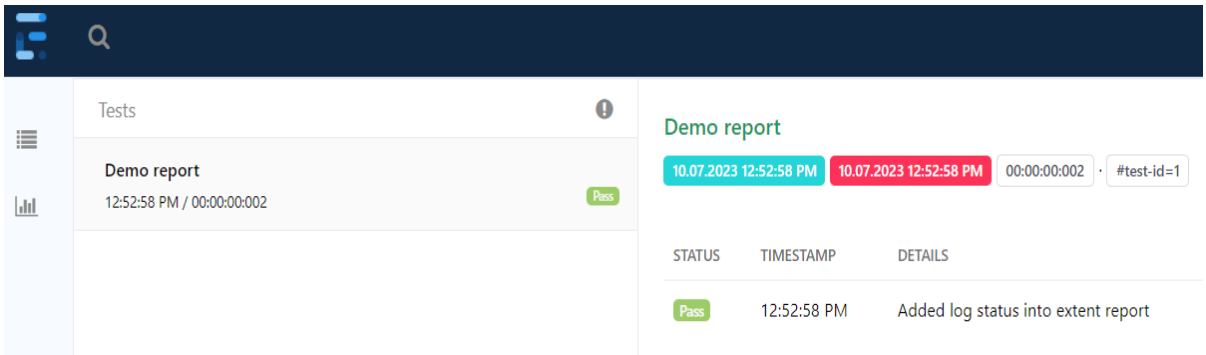
- 1. Go to browser search for " Extent reports".
- 2. Click on first link
- 3. Docs→version 5→click on java.
- 4. Installation→Maven→copy the dependency
- 5. Paste dependency in pom.xml
- 6. Mention version or type LATEST

```
<dependency>
    <groupId>com.aventstack</groupId>
    <artifactId>extentreports</artifactId>
    <version>LATEST</version>
</dependency>
```

Usage of Extent Reports

1. Create an instance of ExtentSparkReporter
    ExtentSparkReporter spark=new ExtentSparkReporter("destination location of reports")
2. Create an instance of ExtentReporter
    ExtentReports report = new ExtentReports();
3. Attach an instance of ExtentSparkReporter to ExtentReports
    report.attachReporter(spark);
4. Create an instance of ExtentTest
    ExtentTest test = report.createTest("testcase name");
5. Use log method to log the status inside extent report
    test.log(Status.PASS , "log message") ;
6. Call flush method to write test information to destination.
    report.flush();

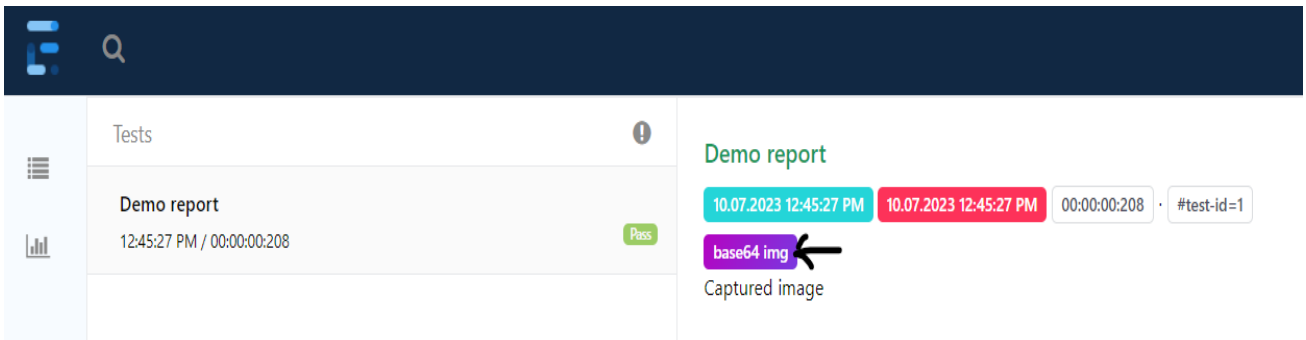
```
public class ToLearnExtentReports {
    @Test
    Run | Debug
    public void reports() {
        ExtentSparkReporter spark=new ExtentSparkReporter("./reports/DemoReport.html");
        ExtentReports report=new ExtentReports();
        report.attachReporter(spark);
        ExtentTest test = report.createTest("Demo report");
        test.log(Status.PASS, "Added log status inside extent report");
        report.flush();
    }
}
```



<u>To attach Screenshot in Extent Reports</u>

Use addScreenCapture method in Extent Test and specify the type of screenshot.

```
public class AttachSSInExtentReport {
    @Test
    Run | Debug
    public void screenshot() {
        WebDriver driver=new ChromeDriver();
        driver.get("https://demo.actitime.com/login.do");
        TakesScreenshot ts=(TakesScreenshot) driver;
        ExtentSparkReporter spark=new ExtentSparkReporter("./reports/DemoReport.html");
        ExtentReports report=new ExtentReports();
        report.attachReporter(spark);
        ExtentTest test = report.createTest("Demo report");
        test.addScreenCaptureFromBase64String(ts.getScreenshotAs(OutputType.BASE64),"Captured image");
        report.flush();
    }
}
```



Coding Standards

- Code standards are rules, techniques, and guidelines to create cleaner, better readable, and more efficient code with minimal bugs and errors.

Why automation engineer should follow coding standards?

- We know huge IT market running on open-source technologies and open-sources require coding experience as base.
- Every Automation QA should follow programming guidelines because that is one of the parameter to become good quality automation engineer.

Programming Coding standards

1. Write clean code
2. Package name should be: domainName.appName.moduleNameTest
3. Code should be optimized.
4. Remove unused variable and import statements.
5. Provide author for every class.
6. Provide proper comments.

Automation coding standards

- Test data should not be hard coded within the test scripts.
- Xpath should not be absolute.
- Avoid indexing in xpath.
- Locator should not be hard coded.
- Avoid try and catch block in test scripts.
- Avoid sleep statements.

GitHub

- It is a source control tool/version control tool/configuration management tool.
- GitHub is a distributed cloud decentralized repository where we can maintain our source code in one place and its being used by entire project team.
- Difference b/w centralized repository and Decentralized repository

Centralized repository (On premises)	Decentralized repository (Cloud based)
We need to buy physical server	No need to buy physical server
Initial investment is high	Initial investment is low
Scale up and Scale down is difficult	Scale up and Scale down is easy
Ex: SVN, CVS, VSS, Perforce	Ex: GitHub/GitLab/GitBucket

- Developer usage of Git: Used to maintain the source code of the application in one place.
- Automation usage of Git: Used to maintain the entire automation framework in one place.

GitClient

There are 2 softwares in Git Community

1. GitHub :- Used to maintain the source code in one place.
2. GitClient :- It is a software used to communicate to GitHub.
   Ex: EGit, GitBash, GitDesktop

Note :- Eclipse by default supports EGit software.

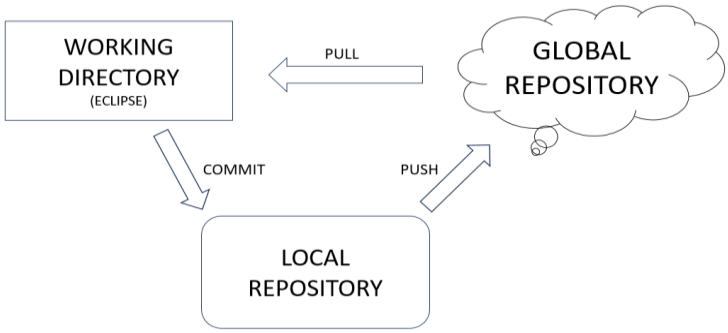
Advantages of GitHub Cloud Repository

- Since it is a cloud-based repo no need to have maintenance team.
- No need of physical location to maintain servers.
- Pay rent for only what you use.
- Can be accessed anywhere using internet and browser.
- Sharing framework between the team is very easy.
- There will be no code conflict.
- No threats for master framework it will be very safe.

GitHub Architecture

There are 3 Stages in Git :

Working Directory→Local Repository→Global Repository



Why Git is Decentralized repository?

- Git is Decentralized repository because in Git before pushing any code to GitHub we have to commit the code to local repository first, make sure code is working in local repository then push code to GitHub(Global repo).

Git Basic Commands

Commit : push code from working directory to local repository.

Push : push code from local repository to global repository.

Pull : get latest update code from global repo to local repo.

Clone : get the entire code base from global repo to local repo for the first time.

Merge : merge the code global repo to local repo.

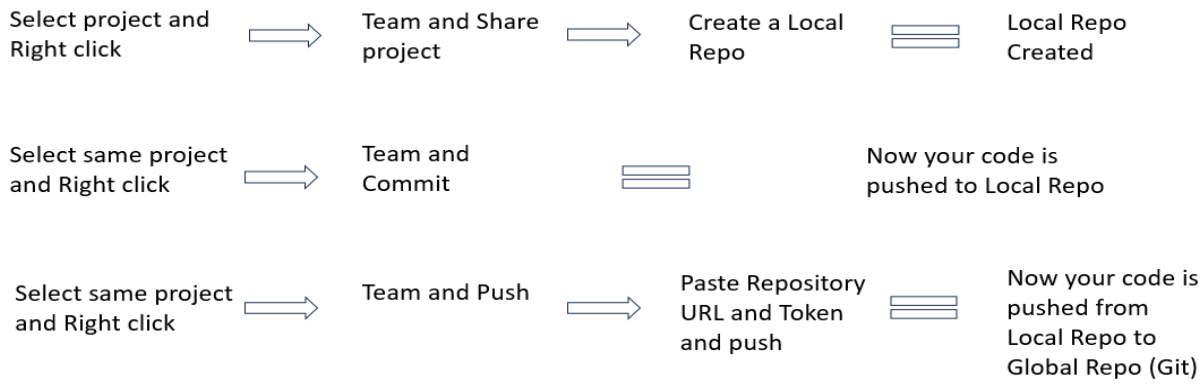
Fork : copy the code from master branch to local branch.

How to create a Repository in Git?

Navigate to GitHub website. If you are a new user, Sign up by giving all the required details.

- Login to account, in Home page click on New.
- Give the Repository Name
- Select Public or Private option based on your requirement.
- Click on Create Repository.
- Repository link will be generated.

How to push project to Git?



What are Tokens and How to Generate?

- Tokens are the keys to access Repository.

How to Generate Tokens?

- In Git Home page, Click on profile icon.
- Click on Settings then Developer Settings.
- Click on Personal Access Token then Tokens(Classic).
- Click on Generate new token (Classic Token).
- Give a note and select expiration period then select the fields which you have to give access and click on Generate Token.
- Copy and save the generated token somewhere because again we can't get that token once if the page is refreshed.

How to pull the project from Git for the first time?

- In Eclipse go to File then import.
- Click on Git and Project from Git
- Click on Clone URI and paste the Repository Link in URL Section(No need of User Name and Token) and click on Next.
- Now choose the Master Branch and click on Next.
- Check the project name, select the required project and finish.

Git Conflict

- If multiple engineers are working on same module and made some changes in same classes. if all of them tries to push the code into Git, there is a confusion created in Git to which one to consider. This is called as Git Conflict.

Solution:

- Each and every time before pushing the code into Git, it is a best practice to pull first and then push the code into Git so that there will be no conflict.

Master and Local Branch

- Repositories are differentiated according to the requirements.
- Master Branch: Since many engineers will be working on a project, master branch access can't be given to everyone. Only the Stake Holder, PM, AL, and Senior Automation Engineers will be given access to master branch.
- Local Branch: Individual repository in Git for individual engineers. Here engineers will FORK the project from master branch. Whatever the changes made by individuals will be in local branch itself. This will not affect the master branch. After changing the code, individual engineers will place a PULL REQUEST, which will be reviewed by AL and merged into master branch so that master branch will be very safe.

FORK and Pull Request

- Coping the complete Framework from the master branch to local branch is FORK.
- Since Automation Engineer(AE) don't have access to master branch, local branch will be considered as master branch for AE.

- Whatever the changes made by AE is updated to his local branch then from there he will send a PULL REQUEST.
- The request made by Automation Engineer to review the code and merge with the master branch is known as PULL REQUEST.
- Once after the pull request is created it will be reviewed by Automation Lead and if everything is fine it will be merged to master branch.

Jenkins

- It is an open-source CI/CD tool.

CI : Continuous Integration

CD : Continuous Deployment

- It is mainly used by the DevOps Engineers for monitoring the build.

Why Jenkins required in Agile processes?

Jenkins used for Continuous integration(Continuous testing) of test scripts.

Installation steps:

a) Download Jenkins and install:

<u>Desktop installation:-</u>

1. Go to browser
2. search for "download Jenkins" and click on 1st link
3. Click on Windows
4. Jenkins.msi file will be downloaded install→run in local→next→Select jdk 17

<u>Browser level installation:-</u>

1. Enter url = http://localhost:8080
2. for password Copy location and type "win+R" paste it.
3. Copy password and paste it
4. Install suggested plugin
5. provide username, password and email.

b) Precondition 1 : Login to Jenkins & install below Plugin

- Jenkins ➔ Manage Jenkins ➔ Plugins➔ Select "Available Plugins"
- Search & install below Plugins
- Maven Integration Plugin
- GitHub Integration Plugin
- PipeLine- Plugin

c) Precondition 2 : Configure Environment variables in Jenkins for below environments

Jenkins ➔ Manager Jenkins➔ Tools

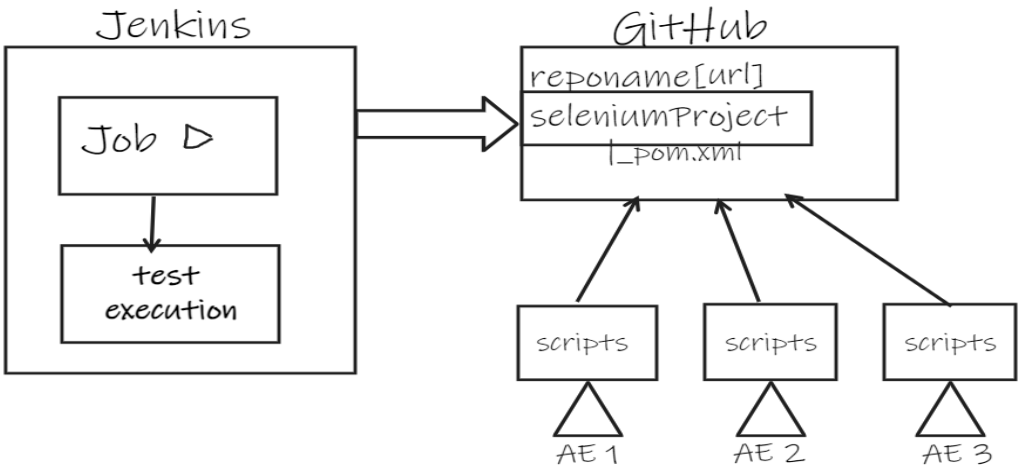
- JDK : C:\Program Files\Java\jdk17
- Maven : D:\apache-maven-3.5.2-bin\apache-maven-3.5.2
- Git : C:\Program Files\Git\bin\git.exe

How to download git.exe?

1. Go to browser and search for “git download”.
2. Click on “Git SCM”.
3. Click on “Windows” and Click on “64-bit Git for windows setup”.
4. Go to downloads then install.

Usage of Jenkins in testing

- Jenkins is a continuous integration tool where continuously monitor the framework Build in git location, then perform appropriate action based on request.



Advantages of Jenkins

1. Jenkins provide 3 level of execution
   a) On demand: Based on customer demand we can start the execution.
   b) On Scheduling: Based on scheduled time, start the execution automatically.
   c) Poll SCM : Where continuously monitor the SCM(GitHub) and automatically start the execution whenever a new build/new test script is getting added to Git.
2. Email Notification:
   - Jenkins sends out an execution Report via email, once execution is completed.
   - Jenkins also Send Build broken Email, when there is a compilation issue between the framework component.
3. Jenkins also support run time parameter.
4. Jenkins also support Pipelines Job.

1. How to create job in Jenkins?

- login to Jenkins
- click on “new Item”
- select “Maven Project” & enter the Job Name
- click on OK button

2. How to LINK selenium Framework which is available in GITHub , to Jenkins ?

- Login to Jenkins
- Get inside the Job
- Click on “Configure” link
- Click “Git” radio button & enter git information (repo url and credentials)
- Scroll down , go to build Division & provide the POM.xml path & goal (Ex: Goals = test)
- Click on “Save” button.

3. How to Execute Jenkins “job”?

- Login to Jenkins
- Get the inside Jenkins job
- Click on “Build now” button

4. How to see the Job Result in Jenkins after the Execution ?

- Get the inside the Job
- Go to build history
- We can see all the Result with time stamp
- click on latest link
- click on “console OutPut”

5. How to Schedule a Job in Jenkins?

- Login to Jenkins
- Get the inside Jenkins job
- Click on configure file
- Click on “Build periodically” checkbox
- Provide the time based on * * * * * separated by space

6. How to configure Poll-SCM execution?

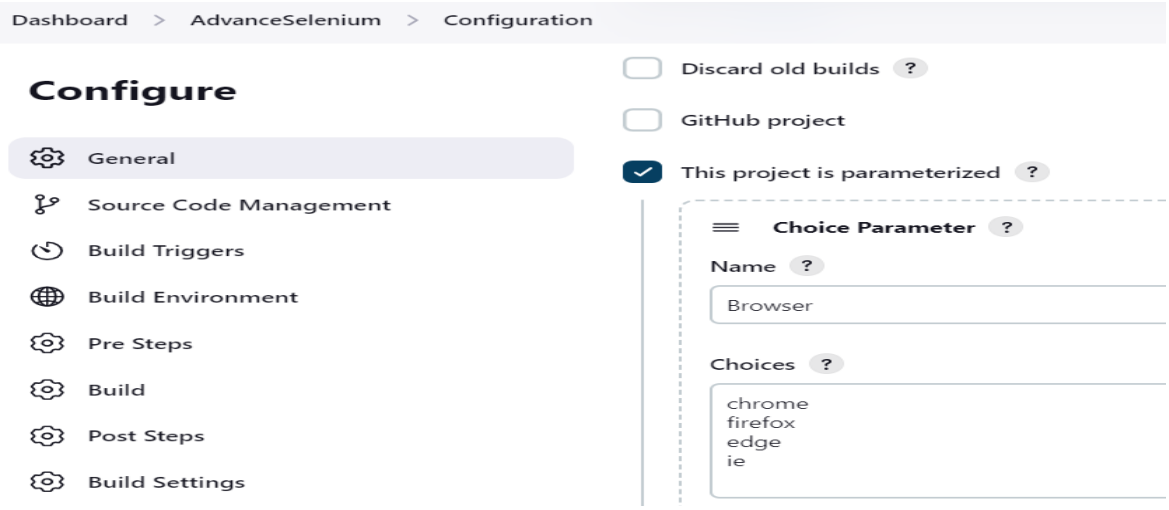
- Login to Jenkins
- Get the inside Jenkins job
- Click on configure file
- Click on “Poll-SCM” checkbox
- Provide the time based on * * * * * separated by space

Jenkins Parameter

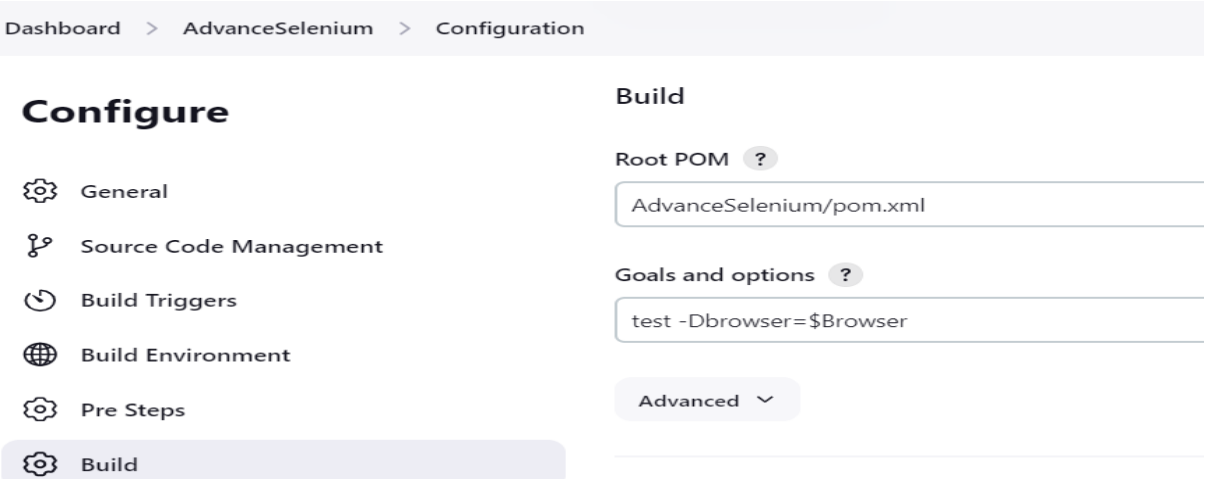
- Jenkins parameter help us to pass data in to test suite in run time from Jenkins.
- Steps to be followed

1. Login to Jenkins
2. Get inside the job
3. Click on “This project is parameterized”
4. Click on add parameter then select “choice parameter” option

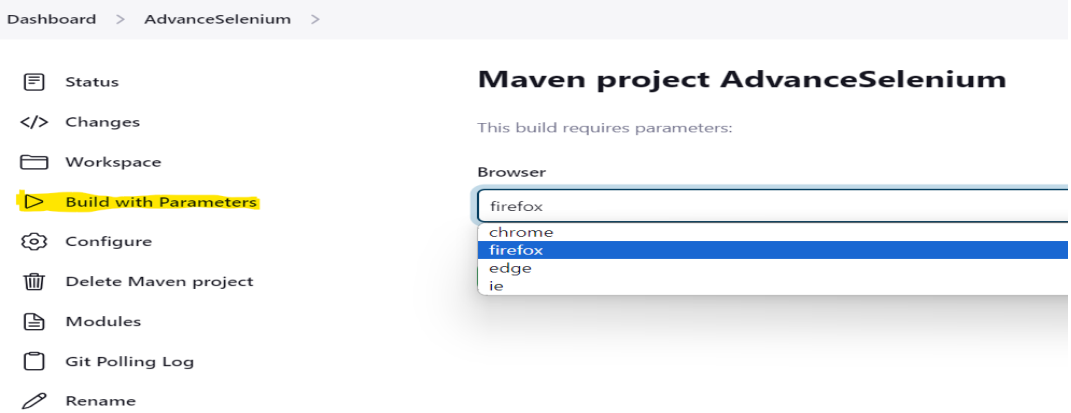
5. Enter the below details.



6. Go to Build Option and edit Goals and options to use Jenkins parameters.



7. Execute Job with “Build with Parameters”



Note:- We should use Maven parameter in selenium scripts.